Packaged Model Instructions

To get the trained model running, you need a standard python environment with lightgbm, pandas and pickle installed. The model expects a single row pandas DF as its input. The DF must have 103 columns in the correct order, marching the API feature list it was trained on(including our 2 custom features total_api_calls and unique_apis_used). All features must be floats. You must pre-fill any missing columns with 0.0 so the structure matches. When you pass this to the mode, it outputs a raw probability score between 0.0 and 1.0. If the score passes the threshold it flags the file as 1(Malware) otherwise it is classified as 0(Benign). You can easily test this setup using the test_predict.py script.

Data Preparation summary

I started by building a custom Python parser in data_processing.py to read the raw LibSVM-style text files line-by-line and extract the threat scores and feature mappings. To make sure identical files didn't bleed across my train and validation sets (which would cause data leakage), I ran a duplicate check on the master DataFrame and dropped any identical rows before doing my splits. I then used feature_name_to_number_mapping.csv to map the numerical column headers to their actual system API names. Because the sparse raw files only list active features, I filled all missing columns with 0.0 to show those APIs weren't used. After that, I engineered two features: total_api_calls (the sum of all API frequencies) and unique_apis_used (counting only columns with values > 0 before appending new features to avoid inflating the count). Finally, I used a stratified 65/15/20 split to split the data into training, validation, and testing sets while keeping the 90/10 benign-to-malware balance intact.

Model Summary

I used a LightGBM Classifier for this project. To ensure the training pipeline is 100% reproducible, I set n_jobs=1 and deterministic=True to force single-threaded, consistent training. I also set is_unbalance=True so LightGBM could adjust to the 90/10 split between benign and malware software. I tuned the decision threshold to be 0.38 using the validation set then evaluated on the test set. The model performed well hitting a malware precision of 96.44% and a malware recall of 90.13%.